module: filepath (System.FilePath)
Hackage: https://hackage.haskell.org/package/filepath-1.4.100.0

System.FilePath

```
type FilePath = String
(-<.>) :: FilePath -> String -> FilePath
(<.>) :: FilePath -> String -> FilePath
(</>) :: FilePath -> FilePath -> FilePath

normalise :: FilePath -> FilePath
equalFilePath :: FilePath -> FilePath -> Bool
makeRelative :: FilePath -> FilePath -> FilePath
isRelative :: FilePath -> Bool
isAbsolute :: FilePath -> Bool
isValid :: FilePath -> Bool
makeValid :: FilePath -> FilePath
hasTrailingPathSeparator :: FilePath -> Bool
addTrailingPathSeparator :: FilePath -> FilePath
dropTrailingPathSeparator :: FilePath -> FilePath

-- Path separator '/'
pathSeparator ::
Char                                    -- POSIX='/'
Windows='\\'
pathSeparators :: [Char]
isPathSeparator :: Char -> Bool

-- $PATH separator ':'
searchPathSeparator ::
Char                            -- POSIX=':' Windows=';'
isSearchPathSeparator :: Char -> Bool
splitSearchPath :: String -> [FilePath]
getSearchPath :: IO [FilePath]

-- Extension
extSeparator ::
Char                                    -- '.'
isExtSeparator :: Char -> Bool
splitExtension :: FilePath -> (String, String)
takeExtension :: FilePath -> String
replaceExtension :: FilePath -> String -> FilePath
dropExtension :: FilePath -> FilePath
addExtension :: FilePath -> String -> FilePath
hasExtension :: FilePath -> Bool
splitExtensions :: FilePath -> (FilePath, String)
dropExtensions :: FilePath -> FilePath
takeExtensions :: FilePath -> String
replaceExtensions :: FilePath -> String -> FilePath
isExtensionOf :: String -> FilePath -> Bool
stripExtension :: String -> FilePath -> Maybe FilePath

-- Directory
splitFileName :: FilePath -> (String, String)
takeFileName :: FilePath -> FilePath
```

```
replaceFileName :: FilePath -> String -> FilePath
dropFileName :: FilePath -> FilePath
takeBaseName :: FilePath -> String
replaceBaseName :: FilePath -> String -> FilePath
takeDirectory :: FilePath -> FilePath
replaceDirectory :: FilePath -> String -> FilePath
combine :: FilePath -> FilePath -> FilePath
splitPath :: FilePath -> [FilePath]
joinPath :: [FilePath] -> FilePath
splitDirectories :: FilePath -> [FilePath]

-- Windows drive 'C:\\'
splitDrive :: FilePath -> (FilePath, FilePath)
joinDrive :: FilePath -> FilePath -> FilePath
takeDrive :: FilePath -> FilePath
hasDrive :: FilePath -> Bool
dropDrive :: FilePath -> FilePath
isDrive :: FilePath -> Bool
```

module: filepath
Library for file path manipulation.

```
---- Basic
--
takeFileName "/a/b/c.txt"        -- "c.txt"
takeBaseName "/a/b/c.txt"        -- "c"
takeDirectory "/a/b/c.txt"       -- "b"
takeExtensions "/a/b/c.txt"      -- ".txt"

"/a/b" </> "c"          -- "/a/b/c"
"/a/b" <.> "txt"        -- "/a/b.txt"
"/a/b.info" <.> "txt"   -- "/a/b.txt"
```

module: directory (Sytem.Directory)
Hackage: https://hackage.haskell.org/package/directory/

System.Directory

```
-- Fixed
getHomeDirectory :: IO FilePath
getAppUserDataDirectory :: FilePath -> IO FilePath
getUserDocumentsDirectory :: IO FilePath
getTemporaryDirectory :: IO FilePath

-- Current Directory
getCurrentDirectory :: IO FilePath
setCurrentDirectory :: FilePath -> IO ()
withCurrentDirectory :: FilePath -> IO a -> IO a

-- Path
exeExtension :: String
```

```
canonicalizePath :: FilePath -> IO FilePath
makeAbsolute :: FilePath -> IO FilePath
makeRelativeToCurrentDirectory :: FilePath -> IO
FilePath

-- Query
listDirectory :: FilePath -> IO [FilePath]
getDirectoryContents :: FilePath -> IO [FilePath]
getFileSize :: FilePath -> IO Integer
doesPathExist :: FilePath -> IO Bool
doesFileExist :: FilePath -> IO Bool
doesDirectoryExist :: FilePath -> IO Bool
findExecutable :: String -> IO (Maybe FilePath)
findExecutables :: String -> IO [FilePath]
findExecutablesInDirectories :: [FilePath] -> String ->
IO [FilePath]
findFile :: [FilePath] -> String -> IO (Maybe FilePath)
findFiles :: [FilePath] -> String -> IO [FilePath]
findFileWith :: (FilePath -> IO Bool) -> [FilePath] ->
String -> IO (Maybe FilePath)
findFilesWith :: (FilePath -> IO Bool) -> [FilePath] ->
String -> IO [FilePath]
isSymbolicLink :: FilePath -> IO Bool

-- Directory
createDirectory :: FilePath -> IO ()
createDirectoryIfMissing :: Bool -> FilePath -> IO ()
removeDirectory :: FilePath -> IO ()
removeDirectoryRecursive :: FilePath -> IO ()
removePathForcibly :: FilePath -> IO ()
renameDirectory :: FilePath -> FilePath -> IO ()
removeFile :: FilePath -> IO ()
renameFile :: FilePath -> FilePath -> IO ()
renamePath :: FilePath -> FilePath -> IO ()
copyFile :: FilePath -> FilePath -> IO ()
copyFileWithMetadata :: FilePath -> FilePath -> IO ()

-- Link
createFileLink :: FilePath -> FilePath -> IO ()
createDirectoryLink :: FilePath -> FilePath -> IO ()
removeDirectoryLink :: FilePath -> IO ()
pathIsSymbolicLink :: FilePath -> IO Bool
getSymbolicLinkTarget :: FilePath -> IO FilePath

-- Permissions
data Permissions
emptyPermissions :: Permissions
readable :: Permissions -> Bool
writable :: Permissions -> Bool
executable :: Permissions -> Bool
searchable :: Permissions -> Bool
setOwnerReadable :: Bool -> Permissions -> Permissions
setOwnerWritable :: Bool -> Permissions -> Permissions
setOwnerExecutable :: Bool -> Permissions -> Permissions
```

```
setOwnerSearchable :: Bool -> Permissions -> Permissions
getPermissions :: FilePath -> IO Permissions
setPermissions :: FilePath -> Permissions -> IO ()
copyPermissions :: FilePath -> FilePath -> IO ()
getAccessTime :: FilePath -> IO UTCTime
getModificationTime :: FilePath -> IO UTCTime
setAccessTime :: FilePath -> UTCTime -> IO ()
setModificationTime :: FilePath -> UTCTime -> IO ()


data XdgDirectory
    = XdgData
    | XdgConfig
    | XdgCache
    | XdgState
getXdgDirectory :: XdgDirectory -> FilePath -> IO
FilePath
data XdgDirectoryList
    = XdgDataDirs
    | XdgConfigDirs
getXdgDirectoryList :: XdgDirectoryList -> IO [FilePath]
```

module: directory
Directory access and manipulation

module: unix-compat (System.PosixCompat)
Hackage: https://hackage.haskell.org/package/unix-compat-0.6

```
getFileStatus :: FilePath -> IO FileStatus
getFdStatus :: FilePath -> IO FileStatus
getSymbolicLinkStatus :: FilePath -> IO FileStatus

deviceID :: FileStatus -> DeviceID
fileID :: FileStatus -> FileID
fileMode :: FileStatus -> FileMode
fileSize :: FileStatus -> FileOffset
fileOwner :: FileStatus -> UserID
fileGroup :: FileStatus -> GroupID
isBlockDevice :: FileStatus -> Bool
isCharacterDevice :: FileStatus -> Bool
isNamedPipe :: FileStatus -> Bool
isRegularFile :: FileStatus -> Bool
isDirectory :: FileStatus -> Bool
isSymbolicLink:: FileStatus -> Bool
isSocket:: FileStatus -> Bool

...
```

module: unix-compat
Library for accessing filesystem

==========
======

System.Random

Hackage: https://hackage.haskell.org/package/random-1.2.1.1/docs/System-Random.html#v:mkStdGen

```
-- System.Random
-- Generators
newtype StdGen = ...
instance RandomGen StdGen

mkStdGen :: Int -> StdGen
initStdGen :: MonadIO m => m StdGen
newStdGen :: MonadIO m => m StdGen
getStdGen :: MonadIO m => m StdGen
setStdGen :: MonadIO m => StdGen -> m ()
getStdRandom :: MonadIO m => (StdGen -> (a, StdGen)) ->
m a

-- Random methods
class RandomGen g where
  {-# MINIMAL split,(genWord32|genWord64|
(next,genRange)) #-}
  genWord8 :: g -> (Word8, g)
  genWord16 :: g -> (Word16, g)
  genWord32 :: g -> (Word32, g)
  genWord64 :: g -> (Word64, g)
  genWord32R :: Word32 -> g -> (Word32, g)
  genWord64R :: Word64 -> g -> (Word64, g)
  genShortByteString :: Int -> g -> (ShortByteString, g)
  split :: g -> (g, g)
  -- deprecated
  next :: g -> (Int, g)
  genRange :: g -> (Int, Int)

uniform :: (RandomGen g, Uniform a) => g -> (a, g)
uniformR :: (RandomGen g, UniformRange a) => (a,a) -> g
-> (a, g)


-- System.Random.Stateful
-- Generators
data StateGenM
instance (RandomGen g, MonadState g m) => StatefulGen
(StateGenM g) m
runStateGen :: RandomGen g => g -> (StateGenM g -> State
g a) -> (a, g)
```

```
runStateGenT :: RandomGen g => g -> (StateGenM g ->
StateT g m a) -> m (a, g)

newtype IOGenM g
instance (RandomGen g, MonadIO m) => StatefulGen (IOGenM
g) m
newIOGenM :: MonadIO m => g -> m (IOGenM g)
applyIOGenM :: MonadIO m => (g -> (a, g)) -> g -> IO a

newtype AtomicGenM g
instance (RandomGen g, MonadIO m) => StatefulGen
(AtomicGenM g) m
newAtomicGenM :: MonadIO m => g -> m (AtomicGenM g)
applyAtomicGenM :: MonadIO m => (g -> (a, g)) -> g -> IO
a

newtype STGenM g s
instance (RandomGen g) => StatefulGen (STGenM g s) (ST
s)
newSTGenM :: g -> ST s  (STGenM g)


-- Random methods
class Monad m => StatefulGen g m where
  {-# MINIMAL (uniformWord32|uniformWord64) #-}
  uniformWord32R :: Word32 -> g -> m Word32
  uniformWord64R :: Word64 -> g -> m Word64
  uniformWord8 :: g -> m Word8
  uniformWord16 :: g -> m Word16
  uniformWord32 :: g -> m Word32
  uniformWord64 :: g -> m Word64
  uniformShortByteString :: Int -> g -> m
ShortByteString

class Uniform a where
  uniformM :: StatefulGen g m => g -> m a
class UniformRange a where
  uniformRM :: StatefulGen g m => (a, a) -> g -> m a
```

Instances

```
instance Uniform Char
instance Uniform Int
instance Uniform Integer
instance Uniform Float
instance Uniform Double
...

instance UniformRange Char
instance UniformRange Int
instance UniformRange Integer
instance UniformRange Float
instance UniformRange Double
...
```

Pseudo-random generator
There are two types of pseudo-random generators:
1) **pure** pseudo-random generator:
i.e. randomness managed **purely**
e.g. using the same generator twice produces the **same** value.
e.g. data **StdGen** is a pure generator
e.g. class **RandomGen** defines methods for using a pure generator

2) **stateful** pseudo-random generator:
i.e. randomness managed **statefully**.
i.e. in general can only be used in a **monad** which can manage state.
e.g. using the same generator twice produces **different** values.
e.g. newtype **IOGenM** is a stateful generator for use in an IO monad
e.g. class StatefulGen defines methods fo r

Builtin generators
- **StdGen**: pure generator
- **IOGenM**: stateful generator in IO
- **AtomicGenM**: stateful generator in IO (thread safe)
- **STGenM**: stateful generator in ST
- **StateGenM**: stateful genrator in a state monad

Basic

```
---- Compute random type
--
g = mkStdGen 1
-- raw random values of RandomGen class
(v, g') = genWord8 g
(v, g') = genWord16 g
(v, g') = genWord32 g
-- class Uniform: any random value
(v, g') = uniform g :: (Bool, StdGen)
(v, g') = uniform g :: (Int, StdGen)
(v, g') = uniform g :: (Char, StdGen)
...

-- class UniformRange: any random value from range
inclusive
(v, g') = uniformR (0,10) g :: (Int, StdGen)        --
random integer from [0,10]
(v, g') = uniformR (0,10) g :: (Float, StdGen)
```

usage: use a **pure** generator **StdGen**+ **pure** seed
i.e. a generator which ALWAYS produces the same value in every run.

```
{-# LANGUAGE TypeApplications #-}
import System.Random
test = do
  let
    -- step: create generator
    g0 = mkStdGen 0

    -- step: generate random values
```

```
    (v1, g1) = uniform @StdGen @Int g0    -- random
integer 1
    (v2, _) = uniform @StdGen @Int g1     -- random
integer 2
  print (v1, v2)                          -- e.g.
(-531973508589804280, -7749304166575005736)


  -- GOTCHA: do NOT use same generator
  -- always the same value
  let
    g0 = mkStdGen 0
    g0' = mkStdGen 0                            -- always
same as g0
    (v1, g1) = uniform @StdGen @Int g0
    (v1', g1') = uniform @StdGen @Int g0      -- same as
v1,g1
    (v1'', g1'') = uniform @StdGen @Int g0'   -- same as
v1,g1
  print (v1, v1', v1'')                       -- e.g.
(-531973508589804280, -531973508589804280,
-531973508589804280) i.e. all same
  print $ g1 == g1'                           -- True
  print $ g1 == g1''                          -- True
```

usage: use a **pure** generator StdGen + **stateful** seed in IO
e.g. initialized by **system state**.

```
{-# LANGUAGE TypeApplications #-}
import System.Random
-- e.g. a generator initialized by IO
test = do
  -- step: initialize generator by system state
  initStdGen >>= setStdGen

  -- step: use generator
  do
    g0 <- getStdGen
    let (v, g0') = uniform @StdGen @Int g0
        (v', g0'') = uniform @StdGen @Int g0'
    setStdGen g0''                            -- update
generator when done
    print (v,v')                              -- e.g.
(-6498802940770954350, -3222410129497034927)

  -- step: use again
  do
    g0 <- getStdGen
    let (v, g0') = uniform @StdGen @Int g0
    setStdGen g0'
    print v                                   -- e.g.
6534011569177982366


  -- e.g. shorthand using getStdRandom
```

```
  newStdGen >>= setStdGen
  v1 <- getStdRandom @IO @Int uniform         -- same as
getStdGen -> uniform -> setStdGen
  v2 <- getStdRandom @IO @Int uniform
  print (v1, v2)                              -- e.g.
(-3637056490349545248, -6364714664871597144)
```

Usage: use a **stateful** generatorin IO
e.g. **IOGenM** and **AtomicGen**

```
{-# LANGUAGE TypeApplications #-}
import System.Random
import System.Random.Stateful
test = do
  -- e.g. use IOGenM
  --
  -- step: get generator
  iog <- newStdGen >>= newIOGenM
  v1 <- uniformM @Int iog            -- compute random
  v2 <- uniformM @Int iog
  print (v1, v2)                     --
e.g. (-8709960573499734046,-5408001509607563708)

  -- e.g. use AtomicGenM
  atomicG <- newStdGen >>= newAtomicGenM
  v1 <- uniformM @Int atomicG
  v2 <- uniformM @Int atomicG
  print (v1, v2)                     --
e.g. (4261714089680333701,7469965457824032180)
```

usage: use a **stateful** generator STGenM in ST

```
{-# LANGUAGE TypeApplications #-}
import System.Random
import System.Random.Stateful
useSTRandom :: (Int, Int)
useSTRandom = runST $ do
  -- step: initialize generator
  g0 <- newSTGenM (mkStdGen 0)

  -- step: generate random values
  v1 <- uniformM @Int g0
  v2 <- uniformM @Int g0             -- v1 != v2
  return (v1,v2)

useSTRandom                    --
e.g. (-7749304166575005736,-531973508589804280)
-- always same value
useSTRandom                    -- same
i.e. (-7749304166575005736,-531973508589804280)
useSTRandom                    -- same
useSTRandom                    -- same
```

Usage: generate **N** random numbers in a monad
Solution: Use **replicateM** to replicate multiple random values

```
{-# LANGUAGE TypeApplications #-}
import System.Random
import System.Random.Stateful

-- Solution 1: use pure generator
-- e.g. StdGen
manyRandomPure :: Int -> IO [Int]
manyRandomPure n = replicateM n (getStdRandom (uniform
@StdGen @Int))

-- Solution 2: use stateful generator
-- e.g. IOGenM
manyRandomStateful :: Int -> IO [Int]
manyRandomStateful n = replicateM n (getStdGen >>=
newIOGenM >>= uniformM @Int)

test = do
  manyRandomPure 5 >>= print        --
e.g. [-8501609843772513024, -8340057418207154609,... ]
  manyRandomStateful 5 >>= print    -- e.g.
[5863096749920932715, 5863096749920932715, ... ]
```

Usage: generate random values for a **user defined type**
Solution: implement **Uniform/UniformR** classes for the type

```
{-# LANGUAGE TypeApplications #-}
import System.Random
import System.Random.Stateful

data MyPair a = MyPair a a deriving (Show, Eq)

-- step: implement Uniform class
instance Uniform a => Uniform (MyPair a) where
  uniformM g = do
    v1 <- uniformM @a g
    v2 <- uniformM @a g
    return (MyPair v1 v2)


test = do
  -- e.g. use with a pure generator StdGen
  do
    g0 <- initStdGen
    p0 <- getStdRandom (uniform @StdGen @(MyPair Int))
 -- e.g. p0=MyPair (-6570709468791945092)
6099454912737010638
    p1 <- getStdRandom (uniform @StdGen @(MyPair Int))
 -- e.g. p1=MyPair 62789332421536883 84
(-7428986451381785673)
    print p0

  -- e.g. use with a stateful generator IOGenM
```

```
  do
    iog <- initStdGen >>= newIOGenM
    p0 <- uniformM @(MyPair Int) iog             -- e.g.
p0=MyPair 2138900343130294182 (-7651766329752955330)
    p1 <- uniformM @(MyPair Int) iog             -- e.g.
p1=MyPair 2598940049966823670 7925235934398752152
    print p0
```